

Mock Interview Homework

1 Overview

This week, you have been paired with one another new member and will be doing mock technical interviews with them!

You will do two mock interviews, one as the interviewer, one as the interviewee (45 min each). This means the homework should take around 90 minutes total. You can find the interview pair and question assignments on this sheet.

Before giving the interview (as the interviewer), learn the problem well (to the point that you can teach and explain it to someone else).

If you have already seen the question you are the interviewee for, let the interviewer know to pick another problem on the list.

2 Logistics

1. Schedule a time (either one 1.5 hour block or two 45 minute blocks) for you and your partner to meet over Zoom.
2. Log into Codebase's LeetCode account (available on Notion) and review the question you are assigned to interview for.
3. Make sure you understand the expectations for the interview (script + rubric).
4. During the interview:
 - a) Record the Zoom call.
 - b) Have the interviewee log into LeetCode and share their screen.
 - c) Follow the provided interview script.
5. After the interview:
 - a) Go through the rubric and give the interviewee scores + feedback.
 - b) One member of the pair should upload the Zoom recording to this Drive folder.

3 Rubric

3.1 Dimension 1 · Problem Comprehension & Clarification

What's Being Evaluated: How quickly and thoroughly the candidate frames the problem, elicits constraints, and restates examples.

- **4 — Exceptional:** Surfaces all constraints and edge-cases unprompted; proposes follow-up examples revealing hidden complexity.
- **3 — Strong / Hire:** Clarifies key inputs and outputs and at least one edge-case; asks logical questions.
- **2 — Mixed / Leaning No-Hire:** Needs multiple prompts to articulate basic constraints; misses obvious edge-cases.
- **1 — Deficient:** Starts coding without confirming understanding; significant misinterpretation persists.

3.2 Dimension 2 · Algorithmic Approach & Trade-offs

What's Being Evaluated: Quality of the high-level solution, evidence of CS fundamentals, and exploration of alternatives.

- **4 — Exceptional:** Identifies optimal asymptotic solution and at least one plausible alternative; reasons clearly about trade-offs in time, space, and simplicity.
- **3 — Strong / Hire:** Arrives at an asymptotically optimal or near-optimal solution with coherent reasoning.
- **2 — Mixed / Leaning No-Hire:** Produces a workable but non-optimal approach; trade-off discussion is superficial or incorrect.
- **1 — Deficient:** Cannot articulate a complete approach; relies on brute force without justification.

3.3 Dimension 3 · Coding Implementation

What's Being Evaluated: Code correctness, readability, idiomatic style, and use of data structures and APIs.

- **4 — Exceptional:** Produces nearly bug-free code on first pass; uses clear names, modular helpers, idiomatic constructs; layout is crisp and legible.
- **3 — Strong / Hire:** Correct solution after a few minor fixes; names, spacing, and structure are mostly clear.
- **2 — Mixed / Leaning No-Hire:** Compiles only after heavy interviewer support; repetitive, unclear, or non-idiomatic style.
- **1 — Deficient:** Does not complete core logic; syntax and structural mistakes dominate.

3.4 Dimension 4 · Testing & Debugging

What's Being Evaluated: Rigor of self-testing and ability to locate and fix issues methodically.

- **4 — Exceptional:** Designs thorough unit-style tests (happy path and edge-cases); self-identifies and fixes bugs without prompting.
- **3 — Strong / Hire:** Tests main path and one edge-case; finds and fixes most errors with light hints.
- **2 — Mixed / Leaning No-Hire:** Minimal tests; relies on interviewer to surface bugs; fixes are ad-hoc.
- **1 — Deficient:** No tests; cannot locate failures even with guidance.

3.5 Dimension 5 · Complexity Analysis

What's Being Evaluated: Ability to analyze and articulate Big-O for time and space.

- **4 — Exceptional:** Quickly derives and explains precise complexity, including dominating terms and memory overhead.
- **3 — Strong / Hire:** Provides correct Big-O for time and space with brief explanation.
- **2 — Mixed / Leaning No-Hire:** Gives partially correct or hand-wavy analysis; omits space or mis-states key terms.
- **1 — Deficient:** Unable to analyze complexity.

3.6 Dimension 6 · Communication & Collaboration

What's Being Evaluated: Clarity of thought, structured narration, and receptiveness to feedback.

- **4 — Exceptional:** Thinks aloud in a clean, logical story; diagrams data flow; checks alignment with interviewer.
- **3 — Strong / Hire:** Explains assumptions and next steps; adapts gracefully to hints.
- **2 — Mixed / Leaning No-Hire:** Disorganized narration or defensive responses.
- **1 — Deficient:** Communication breakdown; interviewer cannot follow reasoning.

3.7 Dimension 7 · Execution & Time Management

What's Being Evaluated: Ability to pace through phases (understand, design, code, test) within 35–40 minutes.

- **4 — Exceptional:** Finishes complete cycle with headroom; priorities are explicit.
- **3 — Strong / Hire:** Completes critical path; minor scope is trimmed or rushed.
- **2 — Mixed / Leaning No-Hire:** Significant sections incomplete or rushed; loses track of time.
- **1 — Deficient:** Stuck in early phase; no runnable solution by end.

3.8 Dimension 8 · Technical Foundations & Fluency

What's Being Evaluated: Breadth of CS concepts shown organically (data structures, recursion, concurrency, language features).

- **4 — Exceptional:** Demonstrates depth beyond role level; introduces advanced but relevant concepts correctly.
- **3 — Strong / Hire:** Employs appropriate APIs and data structures for the task; uses correct terminology.
- **2 — Mixed / Leaning No-Hire:** Misuses or confuses fundamental concepts; needs coaching to select structures.
- **1 — Deficient:** Fundamental gaps that block solution.

3.9 Scoring & Decision Guide

Primary Signal: Dimensions 1–6

Scale:

- Average score of 3.25 or above with no dimension below 3 → **Strong Hire**
- Average 2.8–3.2 → **Weak Hire (panel discussion)**
- Any dimension scored 1 or overall average below 2.8 → **No-Hire**

4 Interviewer Script

As the interviewer, you are responsible for evaluating the interviewee's problem-solving process. This does not mean you just watch the interviewee attempt the interview; you also should be prepared to give guidance as needed.

Some general guidelines for hints:

- Rather than giving a candidate direct answers, try to probe them with questions.
- Ideally, a candidate will speak aloud their thought process before coding. If they do this, pay attention to their thought process/solution and if there are any potential errors, try to probe them about those before they start coding.
- Feel free to help out with any syntax errors you notice (the point of an interview is to evaluate their logic skills, not lack of typos).

4.1 Opening

Say:

- “Hi, thanks for joining. Let's start with a quick introduction. Can you tell me a bit about yourself?”
- “Today we'll work through a technical problem together. Please think out loud as you go. I may ask clarifying or follow-up questions.”

Present the Problem

- Read the problem clearly.
- “Take a moment to process it. Let me know if you have any clarifying questions.”

If the candidate asks good clarification questions, answer them directly.

If the candidate asks a question which they should figure out (relating to the logic of the interview), ask them what they think to hear them reason through it.

4.2 During the Approach Discussion

Encourage structure:

- “Can you walk me through your thinking?”
- “What would the time and space complexity be?”
- “Is there a more optimal approach?”

If they jump into coding immediately:

- “Before you code, can you briefly explain your approach?”

4.3 During Implementation

If the candidate becomes silent:

- “Talk me through what you’re doing.”

If they are stuck:

- “What data structure might help here?”

Avoid giving away the full solution.

Testing Phase

When they say they are finished:

- “Let’s walk through a test case together.”
- “What edge cases should we test?”

Feedback (After 45 min interview)

Use this format:

1. **Strengths:** “I liked how you . . .”
2. **Improvement Area:** “Your biggest area of improvement is . . .”
3. **Rubric:** Go through each section of the rubric and explain how they scored on each one and why.
4. **Overall Evaluation:** Hire / Lean Hire / Lean No Hire / No Hire (with reasoning)

Be specific, constructive, and professional.

5 Interviewee Script

5.1 Clarification Phase

After hearing the problem:

- “Let me restate the problem to make sure I understand.”

Ask clarifying questions such as:

- “What are the input constraints?”
- “Can the input be empty?”
- “Are values guaranteed to be unique?”
- “What input size should I optimize for?”

5.2 Approach Phase

Start with a simple approach:

- “A straightforward/brute force solution would be . . .”

Analyze complexity:

- “That would be $O(n^2)$ time and $O(1)$ space.”

Then improve:

- “To optimize this, we could use . . .”

Explain tradeoffs clearly. Ideally, you are able to think of 2 potential solutions, describe them clearly, and explain when and/or why solution is better than the other.

Before Coding

- “I’ll implement the optimized approach using . . .”
- Briefly outline major steps before typing. Feel free to outline in pseudocode!

During Coding

Keep communicating:

- “Here I’m initializing . . .”
- “This loop handles . . .”
- “I’m choosing this data structure because . . .”

Testing

Before finishing:

- “Let me walk through an example.”

Test:

- A normal case
- An edge case
- Smallest possible input
- Conceptually large input

Handling Feedback

If challenged:

- “That’s a good point — let me think through that.”
- “We could modify the approach by . . .”

Stay calm, structured, and collaborative.

Contributors

- Jaansi Parsa
- Adelina Chau